

Dewlock - System Architecture

Fail-closed "intent-firewall" copilot for agentic Sui DeFi. Monorepo: `apps/web` (Next.js) + packages `@dewlock/{agent,sui,walrus}`.

High-level flow

```
User -> Copilot (chat) -> intent parse + directive -> Mastra agent (tools)
  +- read tools -> cards (portfolio, swap-options, protocols, user-stats, passport, action-form)
  +- prepareTrade -> Guardian (fail-closed gates) -> unsigned PTB -> WYSIWYS sign (client)
    -> /api/receipt/stream (SSE) -> blob + memwal XP + profile + anchor -> receipt card
```

Guardian (value-move firewall) - `@dewlock/agent/guardian`

Every value move runs through `prepareTrade -> guardianCheck(proposal, suiClient)` (packages/agent/src/guardian.ts:296). It is deterministic CODE, **not** an LLM: the agent only *proposes* an unsigned PTB; the Guardian re-derives the math independently, accumulates `reasons[]/gates[]`, and **any failing gate -> BLOCK** (terminal, no auto-retry). Zero user-fund keys server-side; the user signs the literal artifact that was dry-run (WYSIWYS - signed bytes must equal `approvedDigest` bytes). **Fail-closed on every external dependency**: a missing price, invalid cap config, dry-run/RPC failure, or unknown target all block rather than proceed.

Gate pipeline (executed in this order):

- 1. Allowlist** - every PTB `MoveCall {package::module::function}` must be pre-approved. Runs first so unknown targets are never processed.
- 2. Action-shape** - the PTB's `MoveCall` set must match EXACTLY one declared `actionType` template; closes the "compose two allowlisted calls" bypass (e.g. a swap that smuggles `add_liquidity`). Consequence: **one value action per PTB** - composite/multi-action PTBs are refused by design. Provider-keyed for staking (afSUI vs haSUI shapes are distinct; cross-provider calls block).
- 3. Coin-type provenance** - `coinTypeIn/coinTypeOut` verified on-chain via `CoinMetadata` (anti scam-clone; identity by coin TYPE, never display symbol).
- 4. Injection provenance** - per-field `argProvenance`; a derived recipient (inferred / from recalled memory / pool data) triggers a confirm gate. Anti prompt-injection. (Contact/@mention sends use `user_turn` because the 0x is resolved server-side from the wallet-signed friend book - first-party, not injected data.) Hard-block for borrow/withdraw derived amounts.
- 5. Trusted USD price** - real oracle (live SUI/USD via aggregator, USDC=\$1). No trusted price -> BLOCK (cannot value => cannot verify). LST outflow pricing uses a floor-based formula (independent of provider exchange rate) for afSUI/haSUI.
- 6. Server caps** - per-tx + per-day USD caps (`TX_USD_CAP/DAILY_USD_CAP`; server-authoritative, mainnet-small in prod). Invalid cap config -> block everything. Borrow has a dedicated inflow cap (`BORROW_CAP_USD`).
- 7. SuiNS lookalike** - homograph-normalized edit-distance vs verified contacts (catches 888-1.sui-style spoofs).
- 8. Min-out re-derive** (swaps only) - independently recompute min-output from on-chain decimals + the SAME route source; runs for EVERY swap (not gated on poolId). Anti sandwich / manipulated min-out - the #1 real-money risk.
- 9. Staking constraints** (`checkStakingConstraints`) - LST coin-type provenance against curated map (scam-clone afSUI blocks); minimal-exact per-verb action-shape allowlist (swap/lend cannot ride a stake shape; stake/unstake targets cannot be swapped).
- 10. Post-tx health factor** (borrow/withdraw only, NAVI) - calls NAVI's `getSimulatedHealthFactor` (devInspect, contract-authoritative); blocks when projected HF < threshold (default 1.6) or full-withdraw would leave outstanding debt. Fail-closed on

throw/undefined/Infinity. NAVI-only (Suiend: deposit/repay only). 11. **Orderbook / Lending** - DeepBook: `bm_create` (new BalanceManager), `bm_deposit` (fund it), `limit_order` (POST_ONLY / self-match / expiry / BalanceManager-ceiling), `cancel_order` (resting order), `withdraw_settled` (partial withdrawal, recipient pinned to sender, amount ceilinged by server-recomputed settled balance). Lending: deposit/repay health-improving; `borrow/withdraw` gated by health-factor checks above. 12. **Dry-run + WYSIWYS digest** - dry-run the EXACT PTB bytes including gas for effects verification; compute `approvedDigest = sha256(kindBytes)` over the **TransactionKind only** (programmable inputs + commands; no gas coin, no sender). Guardian returns the kind bytes for the client to sign. Client reconstructs the full tx via `Transaction.fromKind()` and wallet fills gas coin + sender at sign time (fresh on-chain version). WYSIWYS verified by re-deriving the kind digest from wallet-built bytes and asserting equality to `approvedDigest` before execute. Gas/sender excluded from the digest by design - wallet signature verifies signer identity; the kind is what the user consents to. This fixes stale-gas "object ... unavailable for consumption" on single-SUI-coin wallets. 13. **Authoritative value gate** - re-value from the dry-run's ACTUAL net balance deltas (what truly leaves the wallet), re-check caps on that figure, and block when outflow > 1.5x the declared value (`outflow_mismatch`). Catches a PTB that moves more than it declares. 14. **Composite recipe gate** (`checkCompositeRecipe`, atomic single-sign only) - enforces four invariants for declared multi-action recipes (e.g., swap->lend): (a) closed-recipe registry (no ad-hoc composition), (b) target multiset (exact PTB shape), (c) coin-type linkage (output coin matches input coin), (d) delta/owner anti-leak + dual caps (USD cap + net-SUI cap). Any failure -> BLOCK.

Pass -> { `ok:true`, `txBytes`, `approvedDigest`, `preview` }; `preview` (balance deltas + gas + USD value) renders before the confirm button. **Defense-in-depth:** the "moves more than declared" risk is caught independently at gates 1 (allowlist), 2 (shape), and 11 (actual-outflow).

- **Swap gate is signature-based** for aggregator routes: the Cetus aggregator emits per-route, upgradeable integration packages that can't be statically pinned, so swap-route calls are matched by `module::function (router::*, cetus::swap, deepbookv3::swap)` - package-agnostic - bounded by the provider constraint (CETUS+DEEPBOOK) + value gates. `0x2::coin::destroy_zero` (full-balance cleanup) is allowlisted as a zero-value framework call.

Object-ownership classification

The dry-run classifies each outgoing object's destination as: - **you** - assets remaining in the sender's wallet (no-op safety check). - **recipient** - the address the user explicitly designated for this action (e.g., send to a contact, swap-output receiver). Neutral "-> recipient" signal; expected flow. - **third-party** - an address the user never designated (the genuine alarm). E.g., a malicious swap reroute to an attacker's address, a compromised pool that diverts funds. Red [!] alert. - **shared** or **object** - protocol-scoped (pool positions, etc.).

The pre-sign permissions UI surfaces only real third-party transfers as the red alarm; an intentional transfer to its designated recipient is marked neutral (not flagged as suspicious). (Files: `dry-run-object-changes.ts` `classifyOwner`, `dry-run.ts`, `tx-assurance-header.tsx`/`tx-permissions-section.tsx`.)

Intent -> action forms - @dewlock/agent/intent

A deterministic `parseIntent + buildIntentDirective` front-runs the LLM. Self-contained commands ("swap 5 SUI to USDC", "send 2 SUI to 0x...", "stake 10 SUI", "borrow 50 USDC on NAVI") route straight to `prepareTrade`. Missing args ("sell SUI", "send USDC", "lending") route to the `requestActionForm` tool -> an interactive form card; its submit composes the canonical command and re-enters the pipeline (Guardian still gates). Counter-asset defaults: USDC->SUI, else->USDC.

Multi-step chaining - a compound intent ("swap 5 SUI to USDC then lend it on NAVI") is detected via Vietnamese connectors (roi / va sau do / tiep theo) + `isChainableSequence` parser; if valid, routes to a chain-plan card instead of refusal. Each step is a normal single-action PTB; the `PlanStepper` state machine (packages/agent/src/chaining/plan-stepper.ts) handles delta resolution (step k+1 consumes output of step k, not pre-existing balance), stale-object waits (avoid "unavailable for consumption"), and halt semantics (BLOCK at step k cancels later steps). Page refresh loses in-flight chain state (durable resume not yet implemented); a transient stale-object error auto-rebuilds the step with fresh bytes (bounded, never re-sends stale bytes). **Atomic single-sign composite is LIVE** (recipe `swap_lend_v1`): the builder composes swap->lend into ONE PTB via the Cetus aggregator's `routerSwap` (the swap-output coin is fed structurally into the NAVI deposit - no wallet round-trip), an upfront SUI-coverage gate (`assertSuiGasCoverage`) catches shortfalls as `insufficient_gas`, and the Guardian's `checkCompositeRecipe` re-verifies the whole PTB before one signature. On any build/Guardian failure it degrades to the sequential chain - funds and every check unaffected. See `atomic-composite-mode.md`.

Hybrid multi-intent decomposition - the regex parser only splits on then/and/also/plus/, ;&+ (+ VN connectors), so it can't structure complex compounds ("finally"/"." separators, multi-recipient sends, per-clause amounts). When a chainable compound can't be regex-parsed, the turn falls through to an LLM `decomposeIntent` tool: the LLM **proposes** the decomposition as the tool's `steps[]` args (ordered single-action commands + category + `amountFrom`), and the tool's `execute` **deterministically verifies** it fail-closed before any chain renders. The moat holds - the LLM proposes, deterministic code verifies; each step still re-enters the normal single-action pipeline (Guardian + WYSIWYS per step).

```
user message
|
v
detectMultiAction --- single ---> normal single-action pipeline
| multi
v
isChainableSequence? --- no ---> refuse "one action per message"
| yes (swap / lend / stake / send)
v
parseChainSteps (regex)
|--- parsed -----> chain-plan card [fast path, 0 LLM]
|--- null --> LLM proposes steps[] --> verifyDecomposeSteps (fail-closed)
              (decomposeIntent args)      |-- ok --> chain-plan card
              |-- fail --> "send one action at a time"

each step's command --> single-action pipeline --> Guardian --> WYSIWYS signature
```

`verifyDecomposeSteps` (the moat) rejects the WHOLE decomposition on any of: (A) <2 steps; (B) a category not in {swap, lend, stake, send}; (C) `routeAction(command)` disagrees with the declared category (the same router that guards `prepareTrade`); (D) step 0 not `amountFrom="explicit"`; (E) a command that hides a second action (e.g. "swap send all to 0x...") - `CLAUSE_SPLIT_RE` splits on "." `/finally` (never on decimals `0.2` or `SuiNS abc.sui`), so one step is always exactly one action.

SuiNS - @dewlock/sui/suins-resolver

Forward + reverse resolution via the **native JSON-RPC** (resolveNameServiceAddress / resolveNameServiceNames) - NOT @mysten/suins SuinsClient (fails to construct under serverExternalPackages). Bare names auto-resolve (roast2026wc -> .sui); unregistered -> clear block. Reverse-lookup spoof guard + homoglyph lookalike check.

Friend address book - apps/web/lib/contacts + @dewlock/agent/memory/contacts

Per-wallet name->0x book stored as ONE Walrus blob + a memwal pointer (contacts-book: <blobId> @ <ts>, latest-wins; contacts-cleared: tombstone for empty). Pointer ts is monotonic (max(now, prevAt+1)) so latest-wins holds for same-ms writes. The blob is the single source of truth - **no per-contact contact: lines** are written (memwal can't delete them). CRUD via GET/POST/DELETE /api/contacts; every write is **payload-bound wallet-signature** gated (dewlock-contacts:<op>:<wallet>:<ts>:<sha256([op, name, address])>, 5-min freshness) so a captured signature can't be replayed with a swapped address. GET is unauthenticated (exposes the labeled graph - documented tradeoff). Resolution is **deterministic & route-side**: the client passes its freshest book to /api/agent; buildIntentDirective runs matchContacts on a bare-word recipient -> 1 match injects the exact 0x (argProvenance.recipient:"derived"), 2+ -> requestContactPicker card, 0 -> SuiNS. The LLM never supplies a 0x; names are sanitized before entering the directive. Managed from the chat-header "Friend list" dialog + a friend card on the two-column My Dashboard.

Conversation persistence - apps/web/lib/conversations +

apps/web/app/api/conversations

Two stores, clean split. **CONTENT** = one immutable Walrus blob per conversation; messages are Seal-encrypted client-side (enc), the server stores them opaquely. **INDEX** = a per-wallet Upstash Redis HASH convo:idx:<wallet> of {id -> {blobId, titleEnc, createdAt, updatedAt}} - exact key-value (HGETALL/HSET/HDEL/DEL), the source of truth for enumeration. The index used to live in memwal (semantic vector store, ~30-43s lag + shared relay rate limit -> slow/flaky list/read/delete); Redis removed both, so a delete is a plain HDEL with no resurrection/tombstones. **Titles are encrypted client-side** (titleEnc, wallet-derived AES-GCM key - sign-once dewlock-conversation-title-key:v1:<wallet> -> HKDF, cached in memory+localStorage; rendered [gate] Locked until the first sign), so the OPEN list read exposes only ciphertext titles + Seal-protected blobIds - the server stays title-blind. CRUD via GET/POST/DELETE /api/conversations (+ /[id]); every **write** (POST upsert, per-id DELETE, clear-all DELETE) is **session write-auth gated** (dewlock-conversation-auth:<wallet>:<ts>, 30-min token, recovers to wallet) - kills the cross-wallet IDOR. Atomicity is **report-after-HSET**: "saved" only after the Redis index write confirms (an orphan blob expires on Walrus). The index-kv.ts store is **server-only** (REST token never bundled); both UPSTASH_REDIS_REST_* and Vercel's KV_REST_API_* env pairs are accepted. memwal is no longer on the conversation path.

Receipt pipeline - apps/web/lib/workflows/post-action-effects.ts

A plain sequential runner (runPostActionEffectsStreaming, NOT a Mastra workflow object - use await createRun()+run.start() elsewhere). Steps, each bounded + fail-soft: 1. **publish** -> Walrus blob

(immutable receipt) + its on-chain Blob `objectId` (surfaced as the receipt's Sui object when the custom HEAD anchor isn't deployed). ~32s budget (mainnet Walrus is slow). 2. **logAction** -> `memwal action log:` line via `rememberBulk` (queued, ~s - NOT `rememberAndWait` which blocks ~30-43s for indexing). This is the XP source of truth. 3. **updateProfile** -> recompute level/XP/badges, persist monotonically (backstop). 4. **anchor** -> optional on-chain HEAD (operational key only). `POST /api/receipt/stream` runs this and streams steps->per-step->done (SSE) to a progress dialog.

Wallet-switch state isolation

On a genuine wallet switch (A->B or A->logout->B) no wallet's data may surface under another wallet's namespace. Implementation details: - **Conversations** - hard-reset in-memory state + `SessionKey` + title encryption key + conversation write-auth all cleared on `isWalletSwitch` detection. - **Contacts** - friend book cleared *synchronously* before refetch to prevent a stale address book from being fed to the agent (e.g., "send to " resolving an old contact from wallet A). - **Agent stream** - aborted via an `AbortController`; per-line owner guard prevents wallet A's streamed response from appending into wallet B's chat thread. - **Autosave guard** - each thread carries a wallet-STAMPED owner; autosave (1.5s debounce + `visibilitychange/unmount flush`) skips if thread owner != live wallet. The server keys writes by `SIGNER` (the wallet), never by content-owner, so client-side wallet-stamp is the enforcement point. (Files: `use-conversations.ts` + `wallet-switch.ts`, `use-contacts.ts`, `use-copilot-chat.ts`, `app/page.tsx`.)

Memory stack (memwal + Walrus)

`memwal` = `@mysten-incubation/memwal` (relay; eventually-consistent ~30s; NO delete/enumerate API). Categories: `action log`: (XP), `wallet-profile`: (durable profile pointer), `risk cap`: (committed cap, seeded from env), `contacts-book`: (friend-book blob pointer), `token map`: (seeded resolution cache). **Conversations are NOT here** - the index moved to Upstash Redis (see Conversation persistence); `memwal` is scoped to genuine semantic memory only. Walrus blobs hold receipts, profiles, passports, and the friend book (each publish creates an on-chain Blob object). - **`remember()` bounded + fail-soft** - `remember()` no longer blocks unbounded: it was `rememberAndWait` (30-43s indexing block) which on serverless `contacts-book` / pointer write surfaced as "save" errors (especially fresh wallet's first write). Now bounded (~12s timeout) + fail-soft: dispatches the write, stops awaiting after timeout, never throws (late rejection caught + logged). Walrus blob stays authoritative; `memwal` pointer is best-effort. (The receipt log path already uses `rememberBulk` - this extends bounded+fail-soft to `remember()`.) - **Memory page** (`/app` -> Memory): lists global + user categories with signature-gated clear. Clearable: **conversations** (Redis `DEL`; exact count + samples sourced from Redis, titles shown as [gate] since encrypted) and **contacts** (book overwritten/tombstoned, count from the book). `memwal` counts are approximate (recall is semantic); append-only ones (action log) are permanent + shown honestly.

Passport - `@dewlock/agent/memory/passport` + `apps/web/lib/passport`

Per-user identity: level, XP, title, earned badges, action counts, member-since (NOT cap/risk - kept private; NOT volume - structurally \$0). Built **live** from the action log (display authority = `/api/user-stats` / `/api/passport`); persisted as a public Walrus blob + `memwal` pointer + optional on-chain HEAD **out-of-band** (background, diff-gated on level/badge change - never in the awaited receipt request). Surfaced as the Passport card atop My Dashboard with blob + Sui-object proof links + share.

user-stats Redis cache (`apps/web/lib/user-stats/stats-cache.ts` + `lib/redis-client.ts`):

`/api/user-stats` is fronted by a per-wallet Upstash Redis read-through cache (`userstats:<wallet>`, 60s TTL). A hit returns instantly and **identically to every surface** - the dashboard `LevelCard/BadgeGrid` and the copilot `ProfileChatCard` read the same cached value, so level/badges can't disagree between them (the old live-memwal reads were eventually-consistent -> mismatched). The cache is a **mirror of the receipt-derived value** (on-chain receipt log = source of truth), never written with client values. Re-sync on action: the dashboard's post-tx re-polls call `?fresh=1`, which re-derives from the authoritative source and overwrites the cache. Fail-soft: no Redis -> derive live as before. **Known slowness:** the Memory page still recalls memwal live (~5-15s, occasionally empty under rate-limit), so its counts can lag - caching it is a future improvement.

Pre-sign transaction-flow UI features

- **React Flow asset-flow map** - visual diagram of where each coin/asset moves; portaled to body (app-level overlay).
- **Richer flow nodes** - token icons + SuiNS/address sub-lines for senders/recipients.
- **Contracts grouped by protocol** - permissions list organizes Move calls by protocol (Cetus, Aftermath, NAVI, etc.) for clarity.
- **TX-digest chevron toggle** - collapse/expand transaction hash display (replaces hard-to-read underline).
- **Readable sign-error messages** - wrapped error cards with plain-English explanations.
- **Reloaded conversations re-build affordance** - when a conversation is reloaded, signable bytes are never persisted (only the command), so the flow re-runs for a fresh preview; the card shows a "re-build" affordance to let the user verify the new preview before signing.

Key gotchas

- `serverExternalPackages` CJS/ESM interop -> named exports nest under `.default` (aggregator `Env`, `SuinsClient`). Use native RPC / normalized loaders.
- Mastra committed workflow: `await workflow.createRun()` + `run.start({inputData})` - there is NO `createRunAsync`.
- `dist` is loaded via `require()` in routes - re-emit after editing `packages/*/src`.
- Rate-limit per-endpoint `scope` (dev IP is constant "local" -> buckets collide otherwise).